# Hybrid Deep Learning Models (CNN–MLP–RNN) for Fashion-MNIST Classification

# By Asrar Qassem

Overview of what i accomplishment in this project, To create a committee of three diverse deep-learning models using the Fashion MNIST dataset and evaluate their performance both individually and collectively, I follow these steps:

## A. Formation of Diverse Deep-Learning Models

1. **Selection of Models**: Three different neural network architectures are chosen. For diversity, I use:

    - **Convolutional Neural Network (CNN)**: Effective for image data due to its ability to capture spatial hierarchies in images.
    - **Multilayer Perceptron (MLP)**: A basic neural network with fully connected layers. It's less specialized for images but can still provide valuable insights.
    - **Recurrent Neural Network (RNN)** or **Long Short-Term Memory (LSTM)**: Unconventional for image classification, but could capture different patterns in data.
2. **Training**: Each model is trained separately on the Fashion MNIST dataset. This dataset contains grayscale images of fashion products, categorized into 10 classes.

3. **Tuning**: Adjust hyperparameters (like the number of layers, neurons per layer, learning rate, etc.) for each model to ensure they learn differently.

## B. Combining the Three Diverse Models

The three models are combined at the score level through averaging:

1. **Model Diversity**: The committee consists of three different types of neural network architectures:

    - **CNN (Convolutional Neural Network)**: Optimized for image data, capturing spatial hierarchies.
    - **MLP (Multilayer Perceptron)**: A basic fully connected neural network.
    - **RNN (Recurrent Neural Network)**: Processes data sequentially, useful for capturing temporal dynamics, adapted here for image classification.
2. **Score Level Fusion**:

- Each model independently processes the input image and outputs a probability distribution over the possible classes.
- These probability distributions represent the confidence of each model in assigning the input image to each of the possible classes.

3. **Averaging Output Probabilities**:
   - The committee's final prediction for a given input image is obtained by averaging these probability distributions across all models.
   - This means that for each class, the committee calculates the mean of the probabilities assigned to that class by each of the three models.
4. **Decision Rule**:
   - The class with the highest average probability is chosen as the final prediction by the committee.

## C. Evaluation Metrics

To evaluate each model and the committee I used the following metrics:

- **F1-Score**: Harmonic mean of precision and recall, giving a balance between the two.
- **Precision**: Proportion of true positives among all predicted positives.
- **Recall**: Proportion of true positives among all actual positives.
- **Accuracy**: Overall proportion of correct predictions.

## Implementation

1. **Model Training and Evaluation**: Train each model on the Fashion MNIST dataset and evaluate their individual performance using the above metrics.
2. **Committee Evaluation**: Evaluate the committee (ensemble) using the same metrics, but with predictions made by averaging the outputs of the three models.

## Tabulation of Results

The table of Results is created with the following structure:

| Model/Committee | F1-Score | Precision | Recall | Accuracy |
|---|---|---|---|---|
| CNN Model | | | | |
| MLP Model | | | | |
| RNN/LSTM Model | | | | |
| Committee | | | | |

## 1. Import necessary libraries:

I'll use TensorFlow/Keras for building the models and Scikit-learn for evaluation metrics.

In [2]:
```
!pip install tensorflow
```

```
Collecting tensorflow
  Downloading tensorflow-2.21.0-cp313-cp313-macosx_12_0_arm64.whl.metadata
(4.4 kB)
Collecting absl-py>=1.0.0 (from tensorflow)
  Downloading absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse>=1.6.0 (from tensorflow)
  Downloading astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers>=25.9.23 (from tensorflow)
  Downloading flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 (from tensorflow)
  Downloading gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta>=0.1.1 (from tensorflow)
  Downloading google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang>=13.0.0 (from tensorflow)
  Downloading libclang-18.1.1-1-py2.py3-none-macosx_11_0_arm64.whl.metadata
(5.2 kB)
Collecting opt_einsum>=2.3.2 (from tensorflow)
  Downloading opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: packaging in /opt/miniconda3/lib/python3.13/s
ite-packages (from tensorflow) (25.0)
Requirement already satisfied: protobuf<8.0.0,>=6.31.1 in /opt/miniconda3/li
b/python3.13/site-packages (from tensorflow) (7.34.1)
Requirement already satisfied: requests<3,>=2.21.0 in /opt/miniconda3/lib/py
thon3.13/site-packages (from tensorflow) (2.32.5)
Requirement already satisfied: setuptools in /opt/miniconda3/lib/python3.13/
site-packages (from tensorflow) (80.9.0)
Requirement already satisfied: six>=1.12.0 in /opt/miniconda3/lib/python3.1
3/site-packages (from tensorflow) (1.17.0)
Collecting termcolor>=1.1.0 (from tensorflow)
  Downloading termcolor-3.3.0-py3-none-any.whl.metadata (6.5 kB)
Requirement already satisfied: typing_extensions>=3.6.6 in /opt/miniconda3/l
ib/python3.13/site-packages (from tensorflow) (4.15.0)
Collecting wrapt>=1.11.0 (from tensorflow)
  Downloading wrapt-2.1.2-cp313-cp313-macosx_11_0_arm64.whl.metadata (7.4 k
B)
Collecting grpcio<2.0,>=1.24.3 (from tensorflow)
  Downloading grpcio-1.80.0-cp313-cp313-macosx_11_0_universal2.whl.metadata
(3.8 kB)
Collecting keras>=3.12.0 (from tensorflow)
  Downloading keras-3.14.0-py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: numpy>=1.26.0 in /opt/miniconda3/lib/python3.
13/site-packages (from tensorflow) (2.3.5)
Collecting h5py<3.15.0,>=3.11.0 (from tensorflow)
  Downloading h5py-3.14.0-cp313-cp313-macosx_11_0_arm64.whl.metadata (2.7 k
B)
Collecting ml_dtypes<1.0.0,>=0.5.1 (from tensorflow)
  Downloading ml_dtypes-0.5.4-cp313-cp313-macosx_10_13_universal2.whl.metada
ta (8.9 kB)
Requirement already satisfied: charset_normalizer<4,>=2 in /opt/miniconda3/l
ib/python3.13/site-packages (from requests<3,>=2.21.0->tensorflow) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /opt/miniconda3/lib/python3.1
3/site-packages (from requests<3,>=2.21.0->tensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /opt/miniconda3/lib/pyt
hon3.13/site-packages (from requests<3,>=2.21.0->tensorflow) (2.6.1)
Requirement already satisfied: certifi>=2017.4.17 in /opt/miniconda3/lib/pyt
hon3.13/site-packages (from requests<3,>=2.21.0->tensorflow) (2025.11.12)
```

```
Requirement already satisfied: wheel<1.0,>=0.23.0 in /opt/miniconda3/lib/pyt
hon3.13/site-packages (from astunparse>=1.6.0->tensorflow) (0.45.1)
Requirement already satisfied: rich in /opt/miniconda3/lib/python3.13/site-p
ackages (from keras>=3.12.0->tensorflow) (14.2.0)
Collecting namex (from keras>=3.12.0->tensorflow)
  Downloading namex-0.1.0-py3-none-any.whl.metadata (322 bytes)
Collecting optree (from keras>=3.12.0->tensorflow)
  Downloading optree-0.19.0-cp313-cp313-macosx_11_0_arm64.whl.metadata (34 k
B)
Requirement already satisfied: markdown-it-py>=2.2.0 in /opt/miniconda3/lib/
python3.13/site-packages (from rich->keras>=3.12.0->tensorflow) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /opt/miniconda3/li
b/python3.13/site-packages (from rich->keras>=3.12.0->tensorflow) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /opt/miniconda3/lib/python3.13/
site-packages (from markdown-it-py>=2.2.0->rich->keras>=3.12.0->tensorflow)
(0.1.2)
Downloading tensorflow-2.21.0-cp313-cp313-macosx_12_0_arm64.whl (223.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 223.8/223.8 MB 7.6 MB/s  0:00:29
m0:00:0100:01
Downloading grpcio-1.80.0-cp313-cp313-macosx_11_0_universal2.whl (12.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.0/12.0 MB 7.3 MB/s  0:00:01 e
ta 0:00:01
Downloading h5py-3.14.0-cp313-cp313-macosx_11_0_arm64.whl (2.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.8/2.8 MB 8.1 MB/s  0:00:00 eta
0:00:01
Downloading ml_dtypes-0.5.4-cp313-cp313-macosx_10_13_universal2.whl (676 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 676.9/676.9 kB 10.2 MB/s  0:00:0
0
Downloading absl_py-2.4.0-py3-none-any.whl (135 kB)
Downloading astunparse-1.6.3-py2.py3-none-any.whl (12 kB)
Downloading flatbuffers-25.12.19-py2.py3-none-any.whl (26 kB)
Downloading gast-0.7.0-py3-none-any.whl (22 kB)
Downloading google_pasta-0.2.0-py3-none-any.whl (57 kB)
Downloading keras-3.14.0-py3-none-any.whl (1.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 9.3 MB/s  0:00:00
Downloading libclang-18.1.1-1-py2.py3-none-macosx_11_0_arm64.whl (25.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 25.8/25.8 MB 7.4 MB/s  0:00:03 e
ta 0:00:01
Downloading opt_einsum-3.4.0-py3-none-any.whl (71 kB)
Downloading termcolor-3.3.0-py3-none-any.whl (7.7 kB)
Downloading wrapt-2.1.2-cp313-cp313-macosx_11_0_arm64.whl (61 kB)
Downloading namex-0.1.0-py3-none-any.whl (5.9 kB)
Downloading optree-0.19.0-cp313-cp313-macosx_11_0_arm64.whl (392 kB)
Installing collected packages: namex, libclang, flatbuffers, wrapt, termcolo
r, optree, opt_einsum, ml_dtypes, h5py, grpcio, google_pasta, gast, astunpar
se, absl-py, keras, tensorflow
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16/16 [tensorflow]6 [tensorflow]
Successfully installed absl-py-2.4.0 astunparse-1.6.3 flatbuffers-25.12.19 g
ast-0.7.0 google_pasta-0.2.0 grpcio-1.80.0 h5py-3.14.0 keras-3.14.0 libclang
-18.1.1 ml_dtypes-0.5.4 namex-0.1.0 opt_einsum-3.4.0 optree-0.19.0 tensorflo
w-2.21.0 termcolor-3.3.0 wrapt-2.1.2
```

In [4]:

```python
import tensorflow as tf
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense, Conv2D, Flatten, MaxPooling2D, LS
from sklearn.metrics import classification_report, accuracy_score
import numpy as bnp
import pandas as pd
from datetime import datetime
from tensorflow import keras
```

## 2. Loading and Preprocessing the Data

Fashion MNIST is a dataset available directly in TensorFlow

In [5]:
```
# Load and preprocess the Fashion MNIST dataset
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.loa
train_images = train_images / 255.0
test_images = test_images / 255.0
train_labels_categorical = to_categorical(train_labels)
test_labels_categorical = to_categorical(test_labels)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-dat
asets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-dat
asets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 3s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-dat
asets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-dat
asets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
```

In [6]:
```
# Reshape data for different models
train_images_cnn = train_images.reshape(-1, 28, 28, 1)
test_images_cnn = test_images.reshape(-1, 28, 28, 1)
train_images_mlp = train_images.reshape(-1, 784)
test_images_mlp = test_images.reshape(-1, 784)
train_images_lstm = train_images.reshape(-1, 28, 28)
test_images_lstm = test_images.reshape(-1, 28, 28)
```

## 3. Model Building:

Building the Models You'll build three different models - CNN, MLP, and LSTM

**CNN:** Consists of convolutional layers followed by max-pooling and fully connected layers.

**MLP:** Consists of densely connected layers.

**LSTM:** Uses LSTM layers, and requires reshaping the input data to fit the temporal sequence model

In [7]:
```
# Define the models
# a. Convolutional Neural Network (CNN)
def build_cnn_model():
    model = Sequential([
        Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
        MaxPooling2D(2, 2),
        Conv2D(64, (3, 3), activation='relu'),
        MaxPooling2D(2, 2),
        Flatten(),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics
    return model

#b. Multilayer Perceptron (MLP)
def build_mlp_model():
    model = Sequential([
        Dense(512, activation='relu', input_shape=(784,)),
        Dense(256, activation='relu'),
        Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics
    return model

# c. Long Short-Term Memory (LSTM)
def build_lstm_model():
    model = Sequential([
        # Input LSTM layer with return_sequences=True to allow the next LSTM
        LSTM(128, return_sequences=True, input_shape=(28, 28)),
        # Dropout for regularization
        tf.keras.layers.Dropout(0.2),

        # Second LSTM layer, can experiment with the number of units
        LSTM(64, return_sequences=True),
        tf.keras.layers.Dropout(0.2),

        # Third LSTM layer, no return_sequences as we do not need to pass se
        LSTM(64),
        tf.keras.layers.Dropout(0.2),

        # Dense layers for classification
        Dense(64, activation='relu'),
        tf.keras.layers.Dropout(0.2),
        Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics
    return model
```

## Combining the Diverse Models

The models are combined at the score level:

1. **Averaging Output Probabilities `committee_predict()`:** Each model outputs a probability distribution over the classes for an input image. The final prediction is obtained by averaging these probabilities across all models.

## Evaluation and Tabulation

1. **Evaluation Metrics `evaluate_model()`:** We'll use F1-score, precision, recall, and accuracy for evaluation.
2. **Individual and Committee Evaluation `evaluate_committee()`:** Each model is evaluated individually, and then the committee's performance is evaluated using the averaged probabilities

In [8]:
```
# Evaluate each model
def evaluate_model(model, test_images, test_labels):
    predictions = model.predict(test_images)
    predictions_classes = np.argmax(predictions, axis=1)
    report = classification_report(test_labels, predictions_classes, output_
    accuracy = accuracy_score(test_labels, predictions_classes)
    return report, accuracy

# Create the committee
def committee_predict(models, test_images_sets):
    predictions = [model.predict(test_images) for model, test_images in zip(
    avg_predictions = np.mean(predictions, axis=0)
    return np.argmax(avg_predictions, axis=1)

# Evaluate the committee
def evaluate_committee(models, test_images_sets, test_labels):
    predictions = [model.predict(test_images) for model, test_images in zip(
    avg_predictions = np.mean(predictions, axis=0)
    avg_predictions_classes = np.argmax(avg_predictions, axis=1)
    report = classification_report(test_labels, avg_predictions_classes, out
    accuracy = accuracy_score(test_labels, avg_predictions_classes)
    return report, accuracy
```

## 4. Training the Models

Train each model separately

### Train the model CNN

In [9]:
```
# logdir="logs/fit/" + datetime.now().strftime("%Y%m%d-%H%M%S")
# tensorboard_callback = keras.callbacks.TensorBoard(log_dir=logdir)

# Train the models
cnn_model = build_cnn_model()
cnn_model.fit(train_images_cnn, train_labels_categorical, epochs=30, validat
# cnn_model.fit(train_images_cnn, train_labels_categorical, epochs=30, valic
```

```
Epoch 1/30
```

```
/opt/miniconda3/lib/python3.13/site-packages/keras/src/layers/convolutional/
base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argu
ment to a layer. When using Sequential models, prefer using an `Input(shape)
` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8316 - loss: 0.4636
- val_accuracy: 0.8788 - val_loss: 0.3361
Epoch 2/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8861 - loss: 0.3114
- val_accuracy: 0.8912 - val_loss: 0.2957
Epoch 3/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9024 - loss: 0.2660
- val_accuracy: 0.8976 - val_loss: 0.2810
Epoch 4/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9148 - loss: 0.2313
- val_accuracy: 0.9050 - val_loss: 0.2606
Epoch 5/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9232 - loss: 0.2058
- val_accuracy: 0.9091 - val_loss: 0.2468
Epoch 6/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9330 - loss: 0.1832
- val_accuracy: 0.9028 - val_loss: 0.2766
Epoch 7/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9392 - loss: 0.1618
- val_accuracy: 0.8982 - val_loss: 0.2981
Epoch 8/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9462 - loss: 0.1444
- val_accuracy: 0.9079 - val_loss: 0.2727
Epoch 9/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9528 - loss: 0.1261
- val_accuracy: 0.9089 - val_loss: 0.2784
Epoch 10/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9576 - loss: 0.1116
- val_accuracy: 0.9118 - val_loss: 0.2752
Epoch 11/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9636 - loss: 0.0970
- val_accuracy: 0.9135 - val_loss: 0.2973
Epoch 12/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9679 - loss: 0.0843
- val_accuracy: 0.9091 - val_loss: 0.3304
Epoch 13/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9716 - loss: 0.0756
- val_accuracy: 0.9070 - val_loss: 0.3426
Epoch 14/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9763 - loss: 0.0652
- val_accuracy: 0.9049 - val_loss: 0.3931
Epoch 15/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9790 - loss: 0.0564
- val_accuracy: 0.9050 - val_loss: 0.4191
Epoch 16/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9800 - loss: 0.0529
- val_accuracy: 0.9038 - val_loss: 0.4006
Epoch 17/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9830 - loss: 0.0459
- val_accuracy: 0.9100 - val_loss: 0.4079
Epoch 18/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9844 - loss: 0.0427
- val_accuracy: 0.9118 - val_loss: 0.4280
Epoch 19/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9857 - loss: 0.0385
- val_accuracy: 0.9122 - val_loss: 0.4296
```

```
Epoch 20/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9866 - loss: 0.0375
- val_accuracy: 0.9114 - val_loss: 0.4734
Epoch 21/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9885 - loss: 0.0316
- val_accuracy: 0.9123 - val_loss: 0.4624
Epoch 22/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9883 - loss: 0.0330
- val_accuracy: 0.9085 - val_loss: 0.4656
Epoch 23/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9902 - loss: 0.0279
- val_accuracy: 0.9022 - val_loss: 0.6065
Epoch 24/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9886 - loss: 0.0316
- val_accuracy: 0.9056 - val_loss: 0.5746
Epoch 25/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9910 - loss: 0.0251
- val_accuracy: 0.9087 - val_loss: 0.5592
Epoch 26/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9904 - loss: 0.0280
- val_accuracy: 0.9060 - val_loss: 0.5696
Epoch 27/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9918 - loss: 0.0235
- val_accuracy: 0.9095 - val_loss: 0.5554
Epoch 28/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9923 - loss: 0.0236
- val_accuracy: 0.9104 - val_loss: 0.6143
Epoch 29/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9912 - loss: 0.0254
- val_accuracy: 0.9059 - val_loss: 0.5883
Epoch 30/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9923 - loss: 0.0223
- val_accuracy: 0.9079 - val_loss: 0.6105
```

Out[9]:

```
<keras.src.callbacks.history.History at 0x340d5c2f0>
```

### Model summary

This summary gives a layer-by-layer description of CNN model architecture, including the type of each layer, its output shape, and the number of parameters it contains

In [10]:

```
cnn_model.summary()
```

**Model: "sequential"**

| Layer (type) | Output Shape | Par |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | |
| conv2d_1 (Conv2D) | (None, 11, 11, 64) | 18 |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64) | |
| flatten (Flatten) | (None, 1600) | |
| dense (Dense) | (None, 128) | 204 |
| dense_1 (Dense) | (None, 10) | 1 |

**Total params:** 675,104 (2.58 MB)
**Trainable params:** 225,034 (879.04 KB)
**Non-trainable params:** 0 (0.00 B)
**Optimizer params:** 450,070 (1.72 MB)

## Train the model MLP

In [11]:
```
# logdir="logs/fit/" + datetime.now().strftime("%Y%m%d-%H%M%S")
# tensorboard_callback = keras.callbacks.TensorBoard(log_dir=logdir)

mlp_model = build_mlp_model()
mlp_model.fit(train_images_mlp, train_labels_categorical, epochs=30, validat
# mlp_model.fit(train_images_mlp, train_labels_categorical, epochs=30, valic
```

```
Epoch 1/30
/opt/miniconda3/lib/python3.13/site-packages/keras/src/layers/core/dense.py:
107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a lay
er. When using Sequential models, prefer using an `Input(shape)` object as t
he first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8238 - loss: 0.4863
- val_accuracy: 0.8563 - val_loss: 0.3912
Epoch 2/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8656 - loss: 0.3645
- val_accuracy: 0.8751 - val_loss: 0.3474
Epoch 3/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8805 - loss: 0.3242
- val_accuracy: 0.8745 - val_loss: 0.3389
Epoch 4/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8872 - loss: 0.3028
- val_accuracy: 0.8827 - val_loss: 0.3183
Epoch 5/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8952 - loss: 0.2816
- val_accuracy: 0.8731 - val_loss: 0.3520
Epoch 6/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8985 - loss: 0.2675
- val_accuracy: 0.8840 - val_loss: 0.3252
Epoch 7/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9040 - loss: 0.2540
- val_accuracy: 0.8841 - val_loss: 0.3222
Epoch 8/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9080 - loss: 0.2429
- val_accuracy: 0.8857 - val_loss: 0.3404
Epoch 9/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9123 - loss: 0.2310
- val_accuracy: 0.8911 - val_loss: 0.3185
Epoch 10/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9171 - loss: 0.2207
- val_accuracy: 0.8943 - val_loss: 0.3041
Epoch 11/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9194 - loss: 0.2120
- val_accuracy: 0.8892 - val_loss: 0.3424
Epoch 12/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9214 - loss: 0.2056
- val_accuracy: 0.8904 - val_loss: 0.3317
Epoch 13/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9237 - loss: 0.1988
- val_accuracy: 0.8861 - val_loss: 0.3693
Epoch 14/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9267 - loss: 0.1919
- val_accuracy: 0.8962 - val_loss: 0.3196
Epoch 15/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9308 - loss: 0.1828
- val_accuracy: 0.8863 - val_loss: 0.3797
Epoch 16/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9320 - loss: 0.1790
- val_accuracy: 0.8914 - val_loss: 0.3426
Epoch 17/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9360 - loss: 0.1685
- val_accuracy: 0.8945 - val_loss: 0.3636
Epoch 18/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9373 - loss: 0.1652
- val_accuracy: 0.8902 - val_loss: 0.3780
Epoch 19/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9388 - loss: 0.1611
- val_accuracy: 0.8915 - val_loss: 0.3618
```

```
Epoch 20/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9407 - loss: 0.1540
- val_accuracy: 0.8857 - val_loss: 0.4057
Epoch 21/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9429 - loss: 0.1465
- val_accuracy: 0.8928 - val_loss: 0.3848
Epoch 22/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9432 - loss: 0.1491
- val_accuracy: 0.8949 - val_loss: 0.3879
Epoch 23/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9456 - loss: 0.1427
- val_accuracy: 0.8893 - val_loss: 0.4190
Epoch 24/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9468 - loss: 0.1374
- val_accuracy: 0.8929 - val_loss: 0.4081
Epoch 25/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9489 - loss: 0.1347
- val_accuracy: 0.8959 - val_loss: 0.3878
Epoch 26/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9508 - loss: 0.1281
- val_accuracy: 0.8928 - val_loss: 0.4224
Epoch 27/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9539 - loss: 0.1223
- val_accuracy: 0.8965 - val_loss: 0.4284
Epoch 28/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9536 - loss: 0.1248
- val_accuracy: 0.8894 - val_loss: 0.4400
Epoch 29/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.9532 - loss: 0.1220
- val_accuracy: 0.8870 - val_loss: 0.4718
Epoch 30/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9550 - loss: 0.1208
- val_accuracy: 0.8941 - val_loss: 0.4531
```

Out[11]:
```
<keras.src.callbacks.history.History at 0x32e00aad0>
```

### Model summary

This summary gives a layer-by-layer description of MLP model architecture, including the type of each layer, its output shape, and the number of parameters it contains

In [12]:
```
mlp_model.summary()
```

**Model: "sequential_1"**

| Layer (type) | Output Shape | Par |
|---|---|---|
| dense_2 (Dense) | (None, 512) | 401 |
| dense_3 (Dense) | (None, 256) | 131 |
| dense_4 (Dense) | (None, 10) | 2 |

**Total params:** 1,607,456 (6.13 MB)

**Trainable params:** 535,818 (2.04 MB)

**Non-trainable params:** 0 (0.00 B)

**Optimizer params:** 1,071,638 (4.09 MB)

## Train the model LSTM

In [13]:
```
# logdir="logs/fit/" + datetime.now().strftime("%Y%m%d-%H%M%S")
# tensorboard_callback = keras.callbacks.TensorBoard(log_dir=logdir)

lstm_model = build_lstm_model()
lstm_model.fit(train_images_lstm, train_labels_categorical, epochs=30, valic
# lstm_model.fit(train_images_lstm, train_labels_categorical, epochs=30, val
```

```
Epoch 1/30
```

```
/opt/miniconda3/lib/python3.13/site-packages/keras/src/layers/rnn/rnn.py:19
9: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a laye
r. When using Sequential models, prefer using an `Input(shape)` object as th
e first layer in the model instead.
  super().__init__(**kwargs)
```

```
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 31s 20ms/step - accuracy: 0.7352 - loss: 0.72
85 - val_accuracy: 0.8223 - val_loss: 0.4948
Epoch 2/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 30s 20ms/step - accuracy: 0.8240 - loss: 0.48
52 - val_accuracy: 0.8221 - val_loss: 0.4570
Epoch 3/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 27s 18ms/step - accuracy: 0.8457 - loss: 0.42
35 - val_accuracy: 0.8633 - val_loss: 0.3754
Epoch 4/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 27s 18ms/step - accuracy: 0.8576 - loss: 0.38
81 - val_accuracy: 0.8613 - val_loss: 0.3619
Epoch 5/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 27s 18ms/step - accuracy: 0.8668 - loss: 0.36
30 - val_accuracy: 0.8745 - val_loss: 0.3395
Epoch 6/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 28s 18ms/step - accuracy: 0.8731 - loss: 0.34
69 - val_accuracy: 0.8816 - val_loss: 0.3357
Epoch 7/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 28s 18ms/step - accuracy: 0.8795 - loss: 0.32
67 - val_accuracy: 0.8810 - val_loss: 0.3275
Epoch 8/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 29s 19ms/step - accuracy: 0.8838 - loss: 0.31
30 - val_accuracy: 0.8751 - val_loss: 0.3392
Epoch 9/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 29s 19ms/step - accuracy: 0.8886 - loss: 0.30
36 - val_accuracy: 0.8887 - val_loss: 0.3014
Epoch 10/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 30s 20ms/step - accuracy: 0.8932 - loss: 0.29
02 - val_accuracy: 0.8862 - val_loss: 0.3049
Epoch 11/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 27s 18ms/step - accuracy: 0.8959 - loss: 0.27
94 - val_accuracy: 0.8891 - val_loss: 0.2975
Epoch 12/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 28s 18ms/step - accuracy: 0.8984 - loss: 0.27
24 - val_accuracy: 0.8923 - val_loss: 0.2953
Epoch 13/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 29s 20ms/step - accuracy: 0.9035 - loss: 0.26
26 - val_accuracy: 0.8961 - val_loss: 0.2907
Epoch 14/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 29s 19ms/step - accuracy: 0.9037 - loss: 0.25
54 - val_accuracy: 0.8976 - val_loss: 0.2875
Epoch 15/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 31s 21ms/step - accuracy: 0.9091 - loss: 0.24
64 - val_accuracy: 0.8955 - val_loss: 0.3033
Epoch 16/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 27s 18ms/step - accuracy: 0.9129 - loss: 0.23
77 - val_accuracy: 0.8984 - val_loss: 0.2818
Epoch 17/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 30s 20ms/step - accuracy: 0.9145 - loss: 0.23
18 - val_accuracy: 0.8942 - val_loss: 0.2976
Epoch 18/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 28s 19ms/step - accuracy: 0.9176 - loss: 0.22
42 - val_accuracy: 0.8898 - val_loss: 0.3130
Epoch 19/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 29s 19ms/step - accuracy: 0.9179 - loss: 0.22
29 - val_accuracy: 0.8971 - val_loss: 0.2911
```

```
Epoch 20/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 29s 19ms/step - accuracy: 0.9196 - loss: 0.21
61 - val_accuracy: 0.9031 - val_loss: 0.2833
Epoch 21/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 26s 18ms/step - accuracy: 0.9235 - loss: 0.20
89 - val_accuracy: 0.8967 - val_loss: 0.2903
Epoch 22/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 28s 19ms/step - accuracy: 0.9258 - loss: 0.20
12 - val_accuracy: 0.9053 - val_loss: 0.2861
Epoch 23/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 28s 19ms/step - accuracy: 0.9267 - loss: 0.19
61 - val_accuracy: 0.9020 - val_loss: 0.2943
Epoch 24/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 28s 19ms/step - accuracy: 0.9284 - loss: 0.19
31 - val_accuracy: 0.9051 - val_loss: 0.2807
Epoch 25/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 29s 19ms/step - accuracy: 0.9303 - loss: 0.18
54 - val_accuracy: 0.9043 - val_loss: 0.2912
Epoch 26/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 27s 18ms/step - accuracy: 0.9334 - loss: 0.18
19 - val_accuracy: 0.9055 - val_loss: 0.2963
Epoch 27/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 28s 18ms/step - accuracy: 0.9352 - loss: 0.17
54 - val_accuracy: 0.9067 - val_loss: 0.2824
Epoch 28/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 28s 19ms/step - accuracy: 0.9361 - loss: 0.17
06 - val_accuracy: 0.9054 - val_loss: 0.2841
Epoch 29/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 27s 18ms/step - accuracy: 0.9358 - loss: 0.17
20 - val_accuracy: 0.9022 - val_loss: 0.3072
Epoch 30/30
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 28s 19ms/step - accuracy: 0.9408 - loss: 0.16
22 - val_accuracy: 0.9079 - val_loss: 0.2908
```

Out[13]:

```
<keras.src.callbacks.history.History at 0x32e00bc50>
```

### Model summary

This summary gives a layer-by-layer description of LSTM model architecture, including the type of each layer, its output shape, and the number of parameters it contains

In [14]:

```
lstm_model.summary()
```

**Model: "sequential_2"**

| Layer (type) | Output Shape | Par |
|---|---|---|
| lstm (LSTM) | (None, 28, 128) | 80 |
| dropout (Dropout) | (None, 28, 128) | |
| lstm_1 (LSTM) | (None, 28, 64) | 49 |
| dropout_1 (Dropout) | (None, 28, 64) | |
| lstm_2 (LSTM) | (None, 64) | 33 |
| dropout_2 (Dropout) | (None, 64) | |
| dense_5 (Dense) | (None, 64) | 4 |
| dropout_3 (Dropout) | (None, 64) | |
| dense_6 (Dense) | (None, 10) | |

**Total params:** 502,880 (1.92 MB)
**Trainable params:** 167,626 (654.79 KB)
**Non-trainable params:** 0 (0.00 B)
**Optimizer params:** 335,254 (1.28 MB)

## 5. Evaluating the Models

I have used Scikit-learn's classification report to evaluate the models

In [16]:
```
import numpy as np
```

In [17]:
```
# evaluate each model and build the reports.
cnn_report, cnn_accuracy = evaluate_model(cnn_model, test_images_cnn, test_l
mlp_report, mlp_accuracy = evaluate_model(mlp_model, test_images_mlp, test_l
lstm_report, lstm_accuracy = evaluate_model(lstm_model, test_images_lstm, te
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 544us/step
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step
```

## 6. Creating and Evaluating the Committee

Average the output probabilities of the three models

In [18]:
```
# Evaluate committee
committee_report, committee_accuracy = evaluate_committee(
    [cnn_model, mlp_model, lstm_model],
    [test_images_cnn, test_images_mlp, test_images_lstm],
    test_labels
)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 497us/step
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step
```

## 7. Tabulating the Results

I used Pandas to create a table manually after printing the results.

In [19]:
```
# Tabulate the results
results = pd.DataFrame({
    "Model": ["CNN", "MLP", "LSTM", "Committee"],
    "F1-Score": [cnn_report["weighted avg"]["f1-score"], mlp_report["weighte
    "Precision": [cnn_report["weighted avg"]["precision"], mlp_report["weigh
    "Recall": [cnn_report["weighted avg"]["recall"], mlp_report["weighted av
    "Accuracy": [cnn_accuracy, mlp_accuracy, lstm_accuracy, committee_accura
})

print(results)
```

```
       Model  F1-Score  Precision  Recall  Accuracy
0        CNN  0.906821   0.907532  0.9066    0.9066
1        MLP  0.888376   0.889720  0.8882    0.8882
2       LSTM  0.902754   0.902734  0.9029    0.9029
3  Committee  0.919070   0.919132  0.9192    0.9192
```

In [20]:
```
results
```

Out[20]:

| | Model | F1-Score | Precision | Recall | Accuracy |
|---|---|---|---|---|---|
| **0** | CNN | 0.906821 | 0.907532 | 0.9066 | 0.9066 |
| **1** | MLP | 0.888376 | 0.889720 | 0.8882 | 0.8882 |
| **2** | LSTM | 0.902754 | 0.902734 | 0.9029 | 0.9029 |
| **3** | Committee | 0.919070 | 0.919132 | 0.9192 | 0.9192 |

### Exmaple of Committee report evaluation

In [21]:
```
committee_report
```

Out[21]:
```
{'0': {'precision': 0.8687872763419483,
  'recall': 0.874,
  'f1-score': 0.8713858424725822,
  'support': 1000.0},
 '1': {'precision': 0.9949443882709808,
  'recall': 0.984,
  'f1-score': 0.9894419306184012,
  'support': 1000.0},
 '2': {'precision': 0.8718466195761857,
  'recall': 0.864,
  'f1-score': 0.8679055750878956,
  'support': 1000.0},
 '3': {'precision': 0.9040307101727447,
  'recall': 0.942,
  'f1-score': 0.9226248775710089,
  'support': 1000.0},
 '4': {'precision': 0.8790983606557377,
  'recall': 0.858,
  'f1-score': 0.868421052631579,
  'support': 1000.0},
 '5': {'precision': 0.9839357429718876,
  'recall': 0.98,
  'f1-score': 0.9819639278557114,
  'support': 1000.0},
 '6': {'precision': 0.7765419615773509,
  'recall': 0.768,
  'f1-score': 0.7722473604826546,
  'support': 1000.0},
 '7': {'precision': 0.9525193798449613,
  'recall': 0.983,
  'f1-score': 0.96751968503937,
  'support': 1000.0},
 '8': {'precision': 0.978021978021978,
  'recall': 0.979,
  'f1-score': 0.9785107446276862,
  'support': 1000.0},
 '9': {'precision': 0.9815950920245399,
  'recall': 0.96,
  'f1-score': 0.9706774519716885,
  'support': 1000.0},
 'accuracy': 0.9192,
 'macro avg': {'precision': 0.9191321509458314,
  'recall': 0.9192,
  'f1-score': 0.9190698448358579,
  'support': 10000.0},
 'weighted avg': {'precision': 0.9191321509458316,
  'recall': 0.9192,
  'f1-score': 0.9190698448358579,
  'support': 10000.0}}
```

### Exmaple of CNN report evaluation

In [22]:
```
cnn_report
```

Out[22]:
```
{'0': {'precision': 0.8469891411648569,
  'recall': 0.858,
  'f1-score': 0.8524590163934426,
  'support': 1000.0},
 '1': {'precision': 0.9949135300101729,
  'recall': 0.978,
  'f1-score': 0.9863842662632375,
  'support': 1000.0},
 '2': {'precision': 0.8868335146898803,
  'recall': 0.815,
  'f1-score': 0.8494007295466389,
  'support': 1000.0},
 '3': {'precision': 0.9008919722497523,
  'recall': 0.909,
  'f1-score': 0.904927824788452,
  'support': 1000.0},
 '4': {'precision': 0.8554336989032901,
  'recall': 0.858,
  'f1-score': 0.8567149276085871,
  'support': 1000.0},
 '5': {'precision': 0.9705304518664047,
  'recall': 0.988,
  'f1-score': 0.979187314172448,
  'support': 1000.0},
 '6': {'precision': 0.7271863117870723,
  'recall': 0.765,
  'f1-score': 0.7456140350877193,
  'support': 1000.0},
 '7': {'precision': 0.9528023598820059,
  'recall': 0.969,
  'f1-score': 0.9608329201784829,
  'support': 1000.0},
 '8': {'precision': 0.9653808110781404,
  'recall': 0.976,
  'f1-score': 0.9706613625062158,
  'support': 1000.0},
 '9': {'precision': 0.9743589743589743,
  'recall': 0.95,
  'f1-score': 0.9620253164556962,
  'support': 1000.0},
 'accuracy': 0.9066,
 'macro avg': {'precision': 0.9075320765990551,
  'recall': 0.9065999999999999,
  'f1-score': 0.906820771300092,
  'support': 10000.0},
 'weighted avg': {'precision': 0.9075320765990549,
  'recall': 0.9066,
  'f1-score': 0.906820771300092,
  'support': 10000.0}}
```

### Print the Accuracy for CNN

In [23]:
```
cnn_report['accuracy']
```

Out[23]: 0.9066

## Exmaple of MLP report evaluation

In [24]:
```
mlp_report
```

Out[24]:
```
{'0': {'precision': 0.8345035105315948,
  'recall': 0.832,
  'f1-score': 0.8332498748122183,
  'support': 1000.0},
 '1': {'precision': 0.9908069458631257,
  'recall': 0.97,
  'f1-score': 0.9802930773117736,
  'support': 1000.0},
 '2': {'precision': 0.7847619047619048,
  'recall': 0.824,
  'f1-score': 0.8039024390243903,
  'support': 1000.0},
 '3': {'precision': 0.8797709923664122,
  'recall': 0.922,
  'f1-score': 0.900390625,
  'support': 1000.0},
 '4': {'precision': 0.8655367231638418,
  'recall': 0.766,
  'f1-score': 0.8127320954907162,
  'support': 1000.0},
 '5': {'precision': 0.9824198552223371,
  'recall': 0.95,
  'f1-score': 0.9659379766141332,
  'support': 1000.0},
 '6': {'precision': 0.696911196911197,
  'recall': 0.722,
  'f1-score': 0.7092337917485265,
  'support': 1000.0},
 '7': {'precision': 0.9204119850187266,
  'recall': 0.983,
  'f1-score': 0.9506769825918762,
  'support': 1000.0},
 '8': {'precision': 0.9729458917835672,
  'recall': 0.971,
  'f1-score': 0.9719719719719719,
  'support': 1000.0},
 '9': {'precision': 0.9691358024691358,
  'recall': 0.942,
  'f1-score': 0.9553752535496958,
  'support': 1000.0},
 'accuracy': 0.8882,
 'macro avg': {'precision': 0.8897204808091843,
  'recall': 0.8882,
  'f1-score': 0.8883764088115302,
  'support': 10000.0},
 'weighted avg': {'precision': 0.8897204808091843,
  'recall': 0.8882,
  'f1-score': 0.8883764088115302,
  'support': 10000.0}}
```

### Exmaple of LSTM report evaluation

In [25]:
```
lstm_report
```

Out[25]:
```
{'0': {'precision': 0.8403614457831325,
  'recall': 0.837,
  'f1-score': 0.8386773547094188,
  'support': 1000.0},
 '1': {'precision': 0.9868951612903226,
  'recall': 0.979,
  'f1-score': 0.9829317269076305,
  'support': 1000.0},
 '2': {'precision': 0.8352941176470589,
  'recall': 0.852,
  'f1-score': 0.8435643564356435,
  'support': 1000.0},
 '3': {'precision': 0.8898635477582846,
  'recall': 0.913,
  'f1-score': 0.9012833168805529,
  'support': 1000.0},
 '4': {'precision': 0.8363095238095238,
  'recall': 0.843,
  'f1-score': 0.8396414342629482,
  'support': 1000.0},
 '5': {'precision': 0.9797979797979798,
  'recall': 0.97,
  'f1-score': 0.9748743718592965,
  'support': 1000.0},
 '6': {'precision': 0.7523316062176166,
  'recall': 0.726,
  'f1-score': 0.7389312977099237,
  'support': 1000.0},
 '7': {'precision': 0.9556650246305419,
  'recall': 0.97,
  'f1-score': 0.9627791563275434,
  'support': 1000.0},
 '8': {'precision': 0.9827586206896551,
  'recall': 0.969,
  'f1-score': 0.9758308157099698,
  'support': 1000.0},
 '9': {'precision': 0.9680638722554891,
  'recall': 0.97,
  'f1-score': 0.9690309690309691,
  'support': 1000.0},
 'accuracy': 0.9029,
 'macro avg': {'precision': 0.9027340899879605,
  'recall': 0.9029,
  'f1-score': 0.9027544799833895,
  'support': 10000.0},
 'weighted avg': {'precision': 0.9027340899879603,
  'recall': 0.9029,
  'f1-score': 0.9027544799833896,
  'support': 10000.0}}
```

## Function to Predict Test Image

I have write this function `predict_test_image()` to predict a test image for each model and for the committee. it has three functional:

1. Predict the class using each individual model (CNN, MLP, RNN).
2. Predict the class using the committee by averaging the probabilities from each model.
3. Display the image and the actual class.

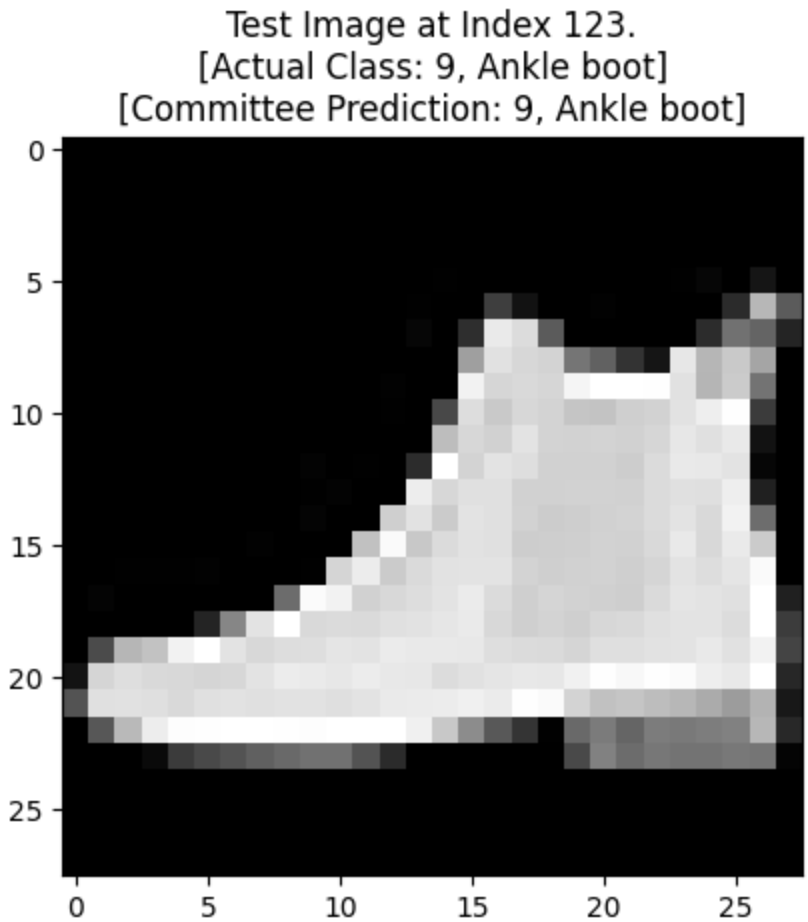
the function has the following parameters:

1. `index` : is the index of the test image in the dataset.
2. `models` : is the list of the three trained models (CNN, MLP, RNN).
3. `test_images_sets` : is a list containing the test images reshaped appropriately for each model.
4. `test_labels` : contains the actual labels of the test images

In [26]:
```
import matplotlib.pyplot as plt

classes={'0':'T-shirt/top',
'1':'Trouser',
'2':'Pullover',
'3':'Dress',
'4':'Coat',
'5':'Sandal',
'6':'Shirt',
'7':'Sneaker',
'8':'Bag',
'9':'Ankle boot'}

def predict_test_image(index, models, test_images_sets, test_labels):
    # Ensure index is within the range
    if index < 0 or index >= len(test_labels):
        print("Index out of range.")
        return

    # Models predictions
    individual_predictions = [model.predict(test_images_sets[i][index:index+
    individual_classes = [np.argmax(pred) for pred in individual_predictions

    # Committee prediction
    avg_prediction = np.mean(individual_predictions, axis=0)
    committee_class = np.argmax(avg_prediction)

    # Actual class
    actual_class = test_labels[index]

    # Display results
    print(f"Predictions: CNN: {individual_classes[0]}- {classes[str(individu
    print(f"Committee Prediction: {committee_class}, {classes[str(committee_
    print(f"Actual Class: {actual_class}, {classes[str(actual_class)]}")

    # Display the image
    plt.imshow(test_images_sets[0][index].reshape(28, 28), cmap='gray')
    plt.title(f"Test Image at Index {index}.\n[Actual Class: {actual_class},
    plt.show()
```

```
# Example usage
predict_test_image(index=123, models=[cnn_model, mlp_model, lstm_model], tes
```

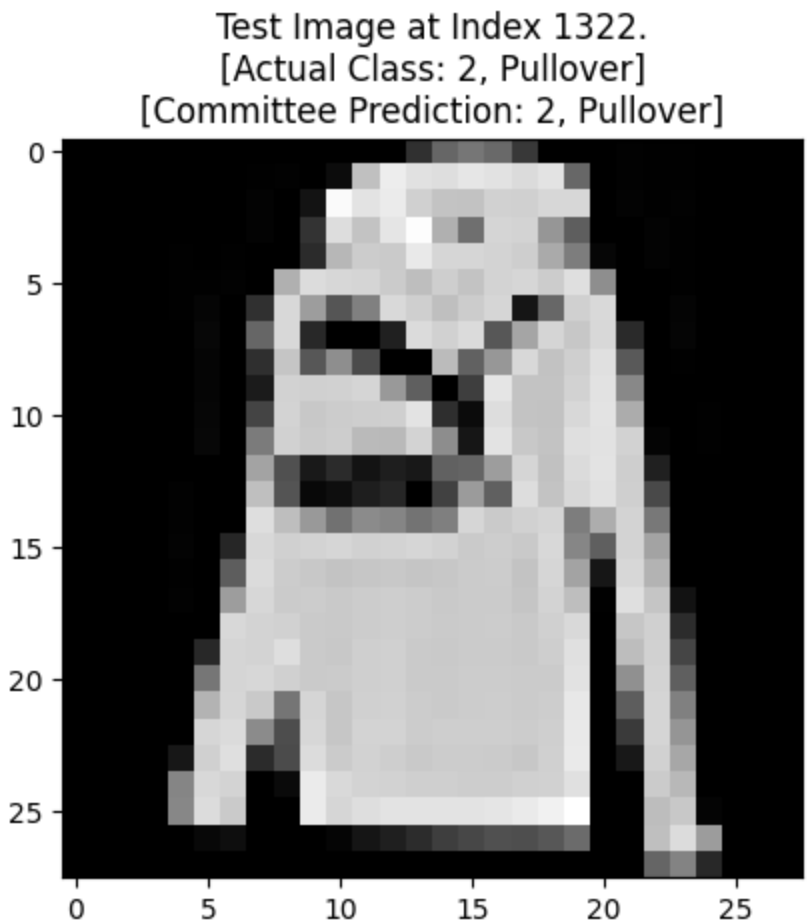
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step
Predictions: CNN: 9- Ankle boot, MLP: 9- Ankle boot, RNN: 9 - Ankle boot
Committee Prediction: 9, Ankle boot
Actual Class: 9, Ankle boot
```




predict the image in the index 1322

In [27]:
```
predict_test_image(index=1322, models=[cnn_model, mlp_model, lstm_model], te
```

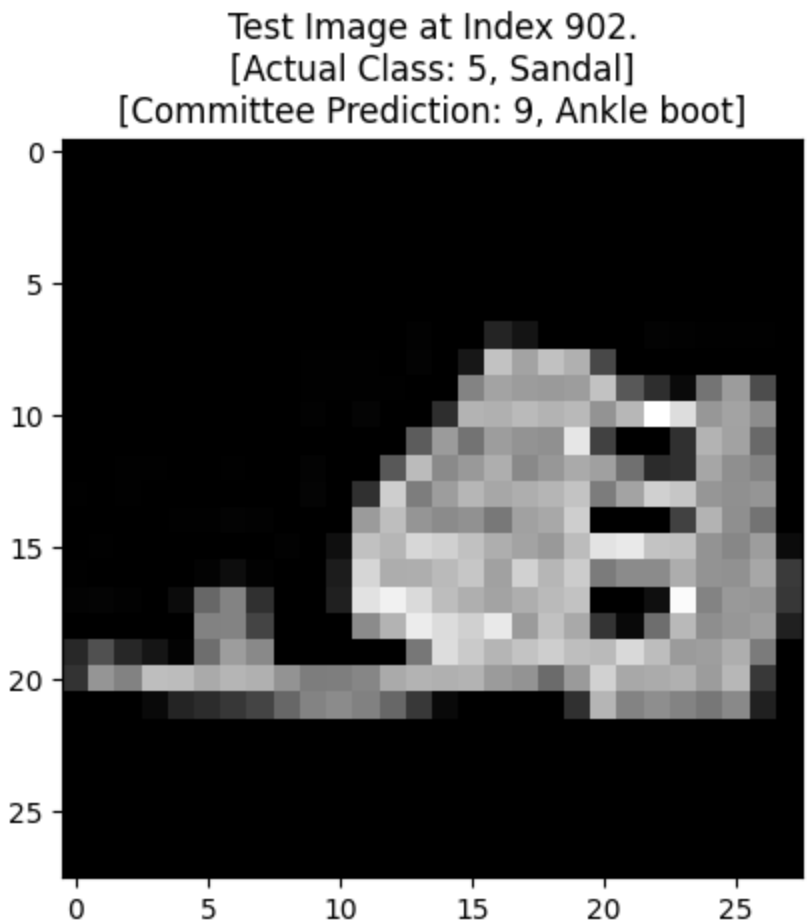
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step
Predictions: CNN: 2- Pullover, MLP: 2- Pullover, RNN: 2 - Pullover
Committee Prediction: 2, Pullover
Actual Class: 2, Pullover
```




predict the image in the index 902

In [28]:
```
predict_test_image(index=902, models=[cnn_model, mlp_model, lstm_model], tes
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step
Predictions: CNN: 5- Sandal, MLP: 9- Ankle boot, RNN: 9 - Ankle boot
Committee Prediction: 9, Ankle boot
Actual Class: 5, Sandal
```




In [31]:
```
!jupyter nbconvert --to pdf hybrid_deep_learning_classifier.ipynb
```